Final GRS Project Report

1) Project Title: Understanding Tail Latency in Kubernetes Microservices

Kubernetes abstracts resource management, but tail latency in microservices often remains unpredictable. This project instruments Kubernetes pods using eBPF to identify kernel-level causes of p99 latency inflation (e.g., scheduling delays, socket buffering, TCP retransmissions). We will correlate pod-level SLO violations with OS- and network-level events and propose mitigation strategies.
Tech: Kubernetes + eBPF

2) Team Members

a) Rajat Kumar Srivastava i) Roll No: MT25078 ii) Mail: rajat25078@iiitd.ac.in
b) Harsha Verma i) Roll No: MT25024 ii) Mail: harsha25024@iiitd.ac.in
c) Anurag Sharma i) Roll No: MT25063 ii) Mail: anurag25063@iiitd.ac.in

3) Problem Statement

Kubernetes abstracts resource management, but tail latency in microservices often remains unpredictable. This project aims to instrument Kubernetes pods using eBPF to identify kernel-level causes of p99 latency inflation (e.g., scheduling delays, socket buffering, TCP retransmissions). Students will correlate pod-level SLO violations with OS- and network-level events and propose mitigation strategies.

4) Motivation — Why is this problem important?

A. **Importance of Tail Latency in Microservices**
As seen in microservice architectures, client queries may traverse several services. High tail latencies may even occur even if the average latency is fine, which impacts users' experience and breaks service-level agreements. Thus, understanding how to address tail latencies is vital rather than focusing on optimizing mean latency alone.

B. **Why Kubernetes Monitoring Alone Is Not Enough**
Kubernetes makes deployment and scalability easier; however, Kubernetes hides all low-level implementation details. Conventional monitoring frameworks provide high-level metrics like CPU, memory usage, and latencies of requests; however, they fail to tell users why latencies are spiking. From observation, P99 latencies may spike due to

low-level kernel operations, including CPU scheduling latency, TCP sending path latencies, retransmissions, and packet losses.

C. **Why This Project Uses eBPF and Controlled Stress**
The eBPF technology enables low overhead tracing of kernel level events during runtime and therefore can be used to diagnose latencies in a near-production environment. In our experiment, we will use a realistic application layer path consisting of Flask requests served to Redis in order to investigate the effects of noisy neighbor in three separate phases of baseline, stress, and recovery.

5) Objectives — What is the scope of your project?

This project focuses on building and evaluating a reproducible Kubernetes-based microservice testbed to study tail latency under controlled contention. The scope is defined through the following objectives:

A. **Build a reproducible 3-node Kubernetes experiment environment**
Set up a Minikube cluster with deterministic node roles so that the same deployment topology can be recreated for repeated measurements and fair comparison across phases.

B. **Enforce a realistic service dependency path per request**
Deploy an application service (Flask) that performs Redis operations for each request, ensuring every HTTP call exercises inter-service communication instead of a static response path.

C. **Execute controlled performance phases**
Run the workload in three structured phases: baseline (no stress), stress (CPU noisy neighbor enabled), and recovery (stress removed), to capture latency behavior under changing system conditions.

D. **Measure latency and throughput across increasing load**
Use wrk-based load generation to collect percentile latency metrics (P50, P75, P90, P99), requests per second, and error counts at multiple connection levels.

E. **Observe kernel-level causes using eBPF instrumentation**
Collect scheduler and network-stack signals such as run-queue delay, TCP send-path latency, retransmissions, and packet-drop indicators using bpftrace probes.

F. **Correlate application tail latency with kernel events**
Analyze whether spikes in high-percentile latency align with kernel-level contention and networking behavior, with emphasis on identifying factors that drive P99 degradation.

G. **Produce a repeatable analysis pipeline for future tuning**

Generate phase-wise and combined result artifacts that can be reused for future experiments, performance tuning, and comparison of mitigation strategies.

6) Background

A. Kubernetes and Microservice Deployment

Kubernetes is an open-source software that manages containerized applications by deploying, scaling, and orchestrating them. The applications are deployed in pods, while services enable network accessibility to those pods. For this specific task, Kubernetes will be deployed in order to establish a test environment for an application service communicating with another data service on the network.

B. Tail Latency and Percentiles

Latency is the time taken to process a request and return a response. Instead of relying only on average latency, percentile metrics are used to capture distribution behavior:

- P50: median latency (typical request)
- P75/P90: higher-latency portion of requests
- P99: tail latency (slowest 1% of requests)

In distributed systems, user experience is often dominated by tail latency rather than average latency, which is why P99 is a key metric in this study.

C. Noisy-Neighbor Effect

In cases where there are clusters that are shared, one workload may impact another due to competing for resources like CPU among others from the host. This is referred to as the “noisy neighbor” problem. The study of this problem is crucial in determining sudden latencies in Kubernetes microservices.

D. Why Redis in the Request Path

A static endpoint would be unrealistic for microservices. For an accurate representation of a dependency graph, each call in the present project is expected to execute Redis commands (SET/GET/INCR). Consequently, there will be actual inter-service communication and backend processing, making the latency measurement more realistic.

E. eBPF and Kernel-Level Observability

eBPF enables safe, low-overhead instrumentation of Linux kernel events without modifying kernel source code. Using bpftrace, probes can be attached to the scheduler and TCP-related events to observe:

- run-queue/scheduling delay,
- TCP send-path latency,
- retransmission, and drop-related behavior.

These signals help explain why application-level tail latency changes under baseline, stress, and recovery conditions, providing deeper insight than application metrics alone.

7) Methodology/Design — What are the components of your system? How do the components interact with each other?

Components of the System

A. **Kubernetes Testbed (3-node Minikube)**
The cluster is configured with fixed node roles to keep placement deterministic across runs: application node, Redis/control node, and stress node.

B. **Application microservice (Flask + Gunicorn)**
The service exposes HTTP endpoints and handles client requests.

C. **Backend data service (Redis)**
Each incoming request performs Redis operations (SET, GET, INCR), forcing real inter-service communication and network traversal.

D. **Load generator (wrk)**
wrk drives controlled concurrent traffic at multiple connection levels to measure latency and throughput behavior.

E. **CPU Stress Workload**
A stress pod based on polinux/stress is launched during Phase 2 on a dedicated stress node to create noisy-neighbor CPU contention.

F. **eBPF observability layer**
bpftrace-based probes run on the host kernel during experiments to capture scheduler, TCP-level behavior, etc.
- Run-queue delay (sched_wakeup/sched_switch based)
- TCP send-path latency (tcp_sendmsg path)
- TCP Retransmit By Process (tcp_retransmit_skb)
- Socket Drop (skb:kfree_skb)
- For Redis (both tcp_send,tcp_recv)

Additional kernel/network indicators from the probe suite

Component Interaction Flow

1. wrk sends concurrent HTTP requests to the Flask application service on Node 2.
2. The Flask service (Node 2) processes each request and performs Redis calls (Master Node).
3. Redis responses return to Flask, and the application sends the final response to the client.

4. In parallel, eBPF probes capture kernel-level events from the scheduler and TCP stack, and other probes are attached.
5. During Phase 2, CPU stress runs on the stress node(NODE 3), increasing contention and affecting scheduling/network timing.
6. Phase-wise measurements (baseline, stress, recovery) are collected and compared to correlate application tail latency with kernel signals.

Experimental Phases

In all Phases, we are generating a high workload through wrk, and Flask is processing those requests and also accessing Redis through inter-node connection.

- **Phase 1: Baseline:** No stress pod. Captures normal latency distribution and kernel behavior.
- **Phase 2: Stress**: CPU stress pod enabled on dedicated stress node. Measures the impact of contention on percentiles and kernel events.
- **Phase 3: Recovery**: Stress pod removed. Measures whether latency and kernel signals return toward baseline.

Design Rationale

- Redis in the request path makes the setup representative of real microservice dependency chains.
- Dedicated stress-node injection isolates contention effects and improves reproducibility.
- Running eBPF alongside the workload allows causal analysis instead of only observing high-level symptoms.
- Phase-based design supports before/during/after comparison for strong tail-latency interpretation.

8) Implementation

The implementation was redesigned from the earlier **single-service** setup into a reproducible **multi-node microservice** experiment focused on tail-latency diagnosis under controlled contention.

A. 3-node Kubernetes provisioning and deterministic node roles

The cluster is provisioned as a 3-node Minikube environment, and each node is labeled for fixed workload placement so experiments are repeatable across runs. Configured placement model:

- Control node: Redis
- App node: Flask service
- Stress node: CPU stress workload (used only in stress phase)

B. Redis deployment pinned to the control node

Redis is deployed with node selection targeting the control-plane node role. This ensures backend placement is stable and preserves the intended cross-node application-to-backend path.

C. **Flask application deployment pinned to the app node with Gunicorn workers**
   The application tier was upgraded from a simple static web target to a Flask service that performs Redis operations per request. It is pinned to the app node and served by Gunicorn workers for concurrent request handling.

D. **CPU stress pod pinned to stress node**
   A dedicated stress pod based on polinux/stress is deployed only during the stress phase and pinned to the stress node. This creates controlled noisy-neighbor CPU contention without changing placement of app/backend services.

E. **3-phase workload runner and CSV result generation**
   Workload execution is scripted into baseline, stress, and recovery phases. Connection levels are fixed, and each run captures percentile latency and throughput metrics into CSV artifacts.

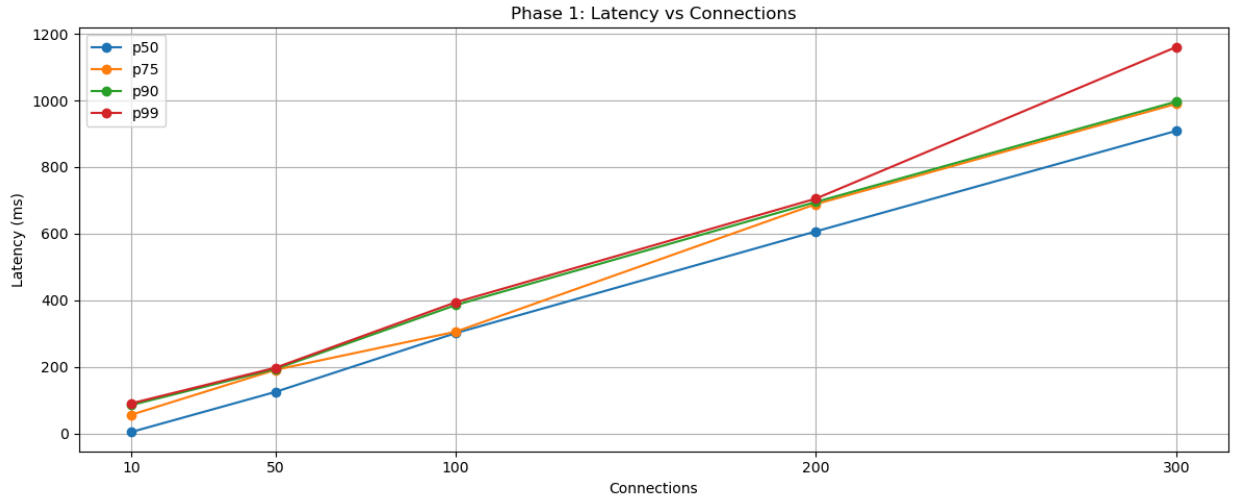
F. **eBPF probe suite used during workload execution**
   Kernel-level observability is collected in parallel with load execution using five probes:
   1. **Run-queue latency probe**: Tracks scheduling delay using scheduler tracepoints.
   2. **TCP send latency probe:** Measures time in TCP send path (tcp_sendmsg).
   3. **TCP retransmit probe:** Counts retransmission events as indicators of network/path stress.
   4. **Socket drop probe:** Tracks packet-drop-related kernel events.
   5. **Redis path RTT approximation probe:** Approximates Redis request/response timing behavior via TCP hooks.

9) Evaluation & observation

Latency vs Load Experiment using wrk at the application layer

PHASE 1- Latency Analysis through wrk request

Connections	P50(ms)	P75(ms)	P90(ms)	P99(ms)
10	5.43	57.17	86.30	91.34
50	125.67	191.91	194.33	198.16
100	301.96	306.15	385.94	394.38
200	607.18	688.99	695.47	705.38
300	908.58	989.64	996.34	1160.00



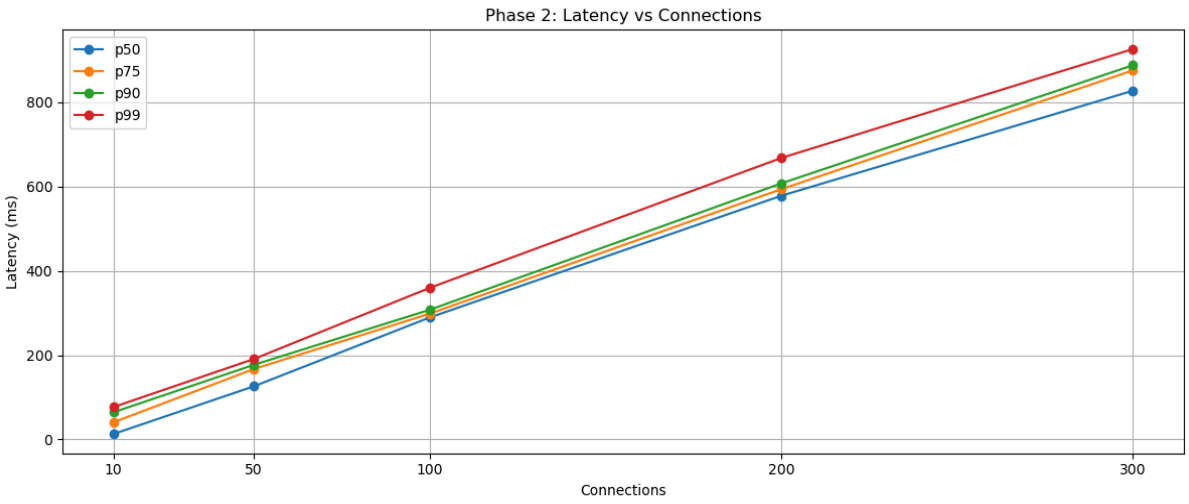
Plot for Latency vs Connections for Phase 1

Observation

- Latency increases almost linearly with connections, indicating normal system scaling under load.
- The gap between P50 and P99 is moderate, meaning tail latency exists but is still controlled.
- At 300 connections, P99 (~1160 ms) becomes significantly higher than median (~900 ms), showing early signs of tail amplification.
- This suggests that even without explicit stress, resource contention begins at higher concurrency levels.
- System is still relatively stable, but queueing delays and scheduling latency are starting to appear.

PHASE 2- Latency Analysis through wrk request

Connections	P50(ms)	P75(ms)	P90(ms)	P99(ms)
10	12.64	40.37	64.22	76.55
50	126.02	167.07	176.96	190.85
100	289.44	298.41	307.58	607.79
200	578.39	593.99	607.79	668.14
300	827.69	875.95	888.20	926.63



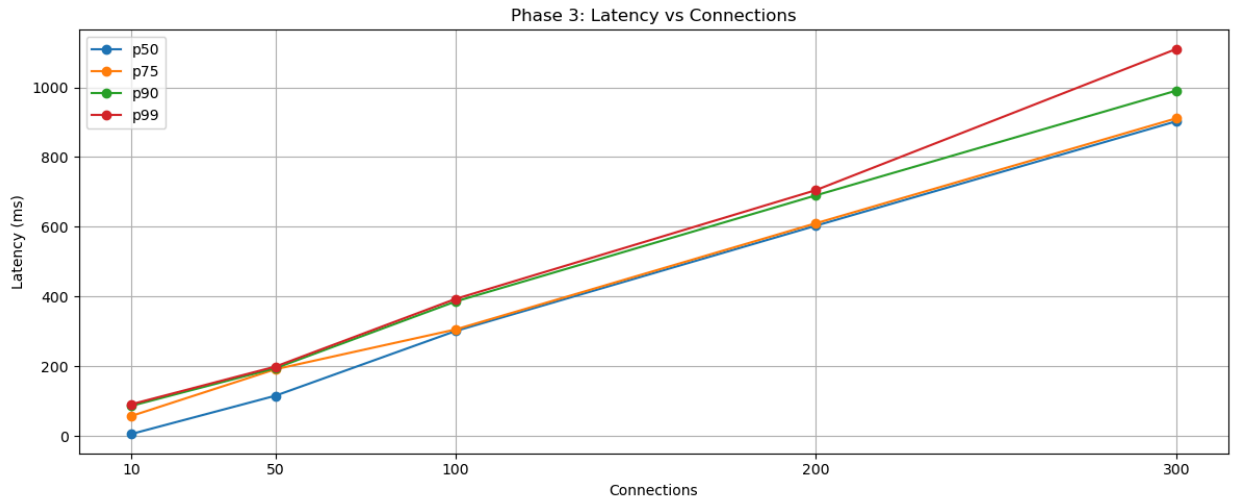
Plot for Latency vs Connections for Phase 2

Observation

- Tail latency (P99) increases disproportionately compared to P50/P75, especially at 100 connections (607 ms spike).
- The gap between percentiles becomes much wider, indicating high variability in request completion times.
- Unlike Phase 1, latency growth is non-linear, showing unstable system behavior.
- Even at moderate load (100–200 connections), tail latency is significantly worse than baseline.
- This clearly indicates resource contention affecting only a subset of requests, not all.

PHASE 3- Latency Analysis through wrk request

Connections	P50(ms)	P75(ms)	P90(ms)	P99(ms)
10	5.48	57.16	86.29	90.76
50	115.31	191.28	194.28	198.93
100	300.92	305.45	385.67	393.06
200	603.13	610.15	690.20	705.21
300	902.63	911.17	990.60	1110.00



Plot for Latency vs Connections for Phase 3

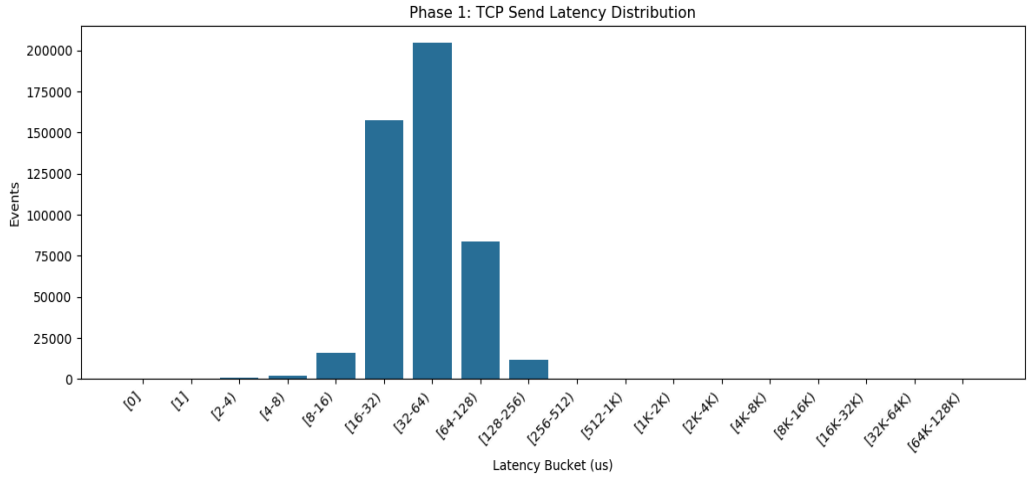
Observation

- Latency trends return closer to Phase 1 behavior, showing system recovery after stress removal.
- However, P99 is still slightly elevated compared to Phase 1 (e.g., ~1110 ms vs ~1160 ms but still high variance).
- The gap between percentiles reduces compared to Phase 2, indicating reduced variability.
- System regains predictable scaling, but not perfectly — suggesting residual effects (e.g., cache, queues, buffers).
- High-load behavior (300 connections) still shows tail latency sensitivity.

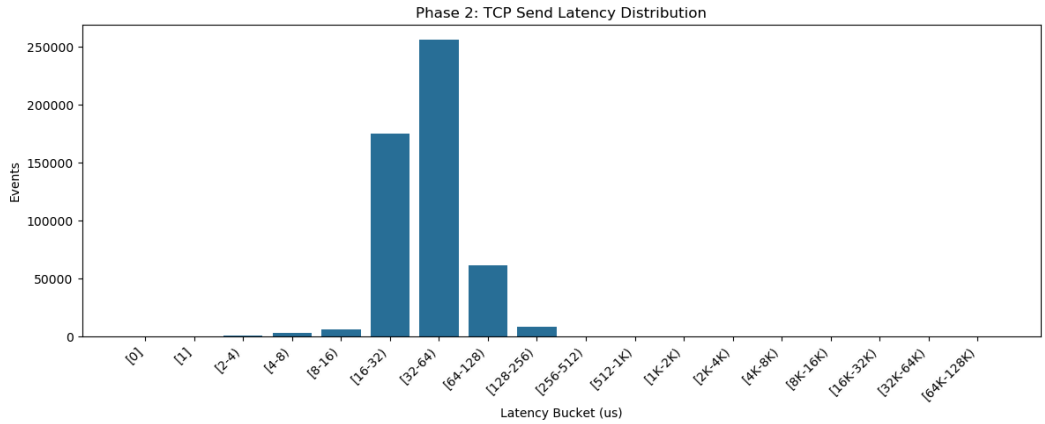
Per-Probe Cross-Phase Comparison

1) TCP Send Latency Probe

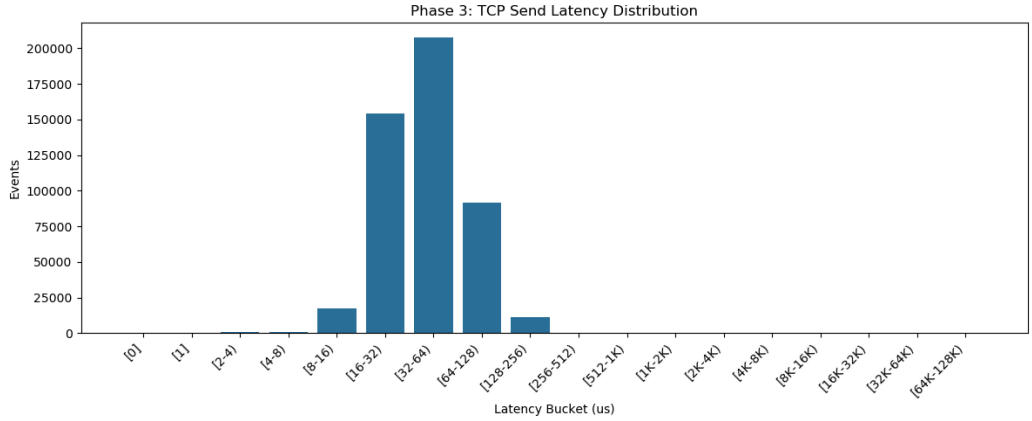
Phase 1: Baseline Plot



Phase 2: Stress Plot



Phase 3: Recovery Plot

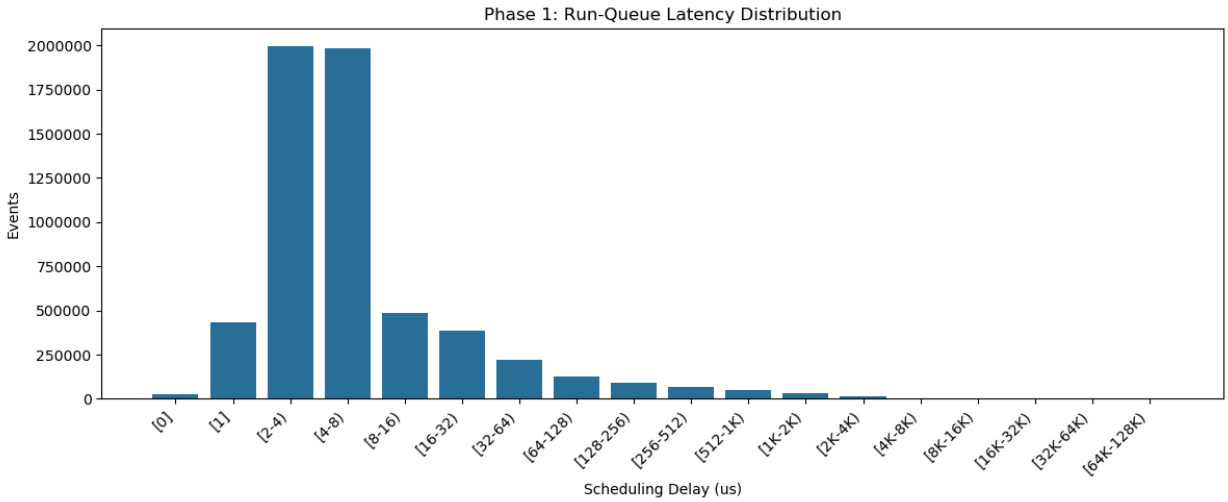


Observation

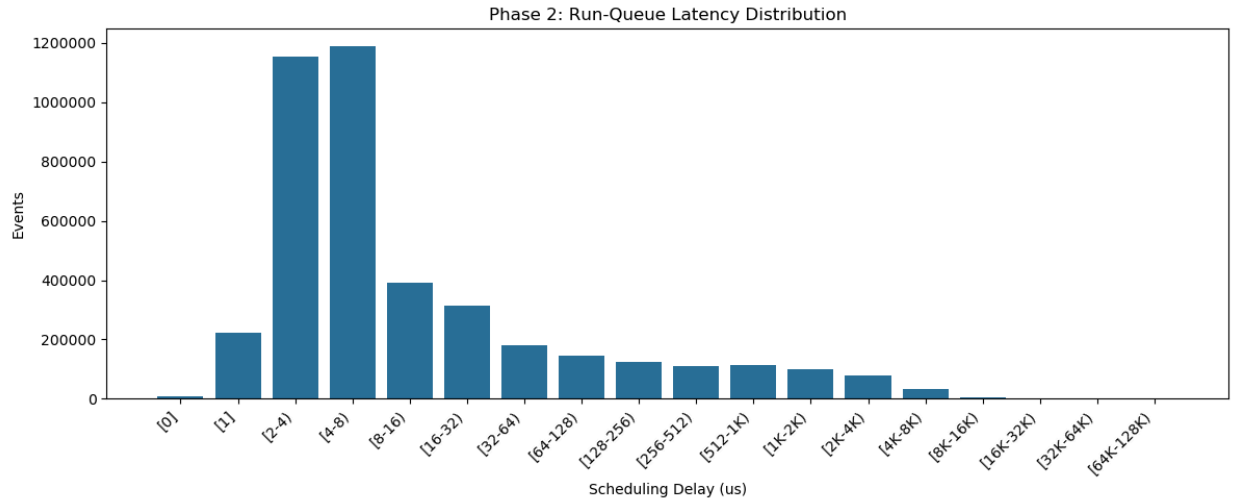
- In Phase 1 (Baseline), most TCP send events are concentrated in lower latency buckets, indicating efficient packet transmission with minimal kernel overhead.
- In Phase 2 (Stress), the distribution shifts toward higher latency buckets, and the spread becomes wider. This shows that TCP send operations are getting delayed under contention, likely due to CPU scheduling delays and network stack congestion.
- The increase in higher latency buckets during stress suggests queue buildup in the socket layer and delayed packet processing, contributing directly to tail latency (p99) spikes observed at the application level.
- In Phase 3 (Recovery), the distribution shifts back toward lower latency buckets, similar to Phase 1, indicating that TCP send latency improves once contention is removed.
- However, a slight presence in higher latency buckets may still remain, suggesting residual buffering or delayed recovery in the network stack

2) CPU Scheduled Latency Probe

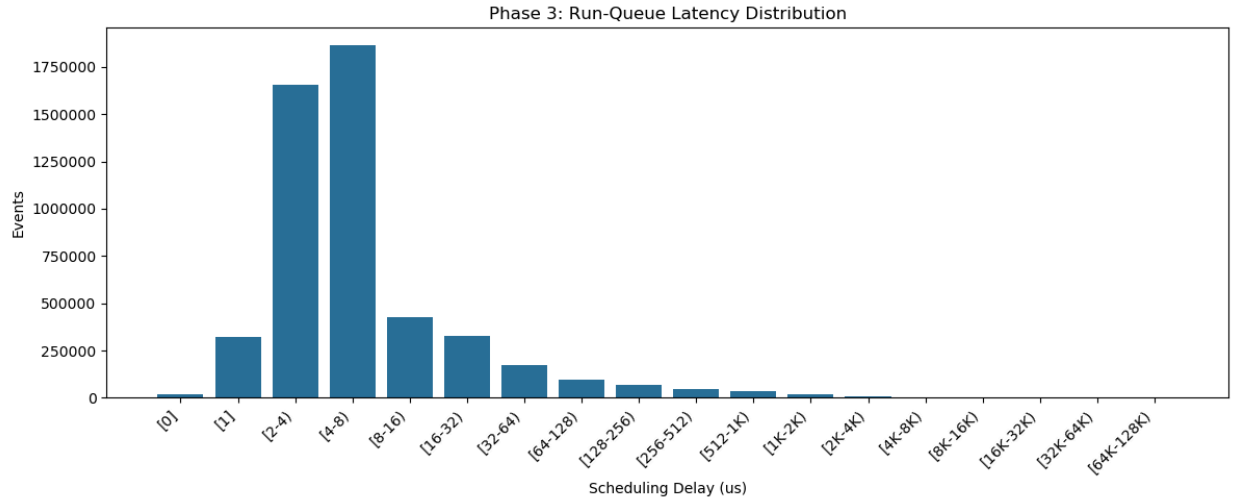
Phase 1: Baseline Plot



Phase 2: Stress Plot



Phase 3: Recovery Plot



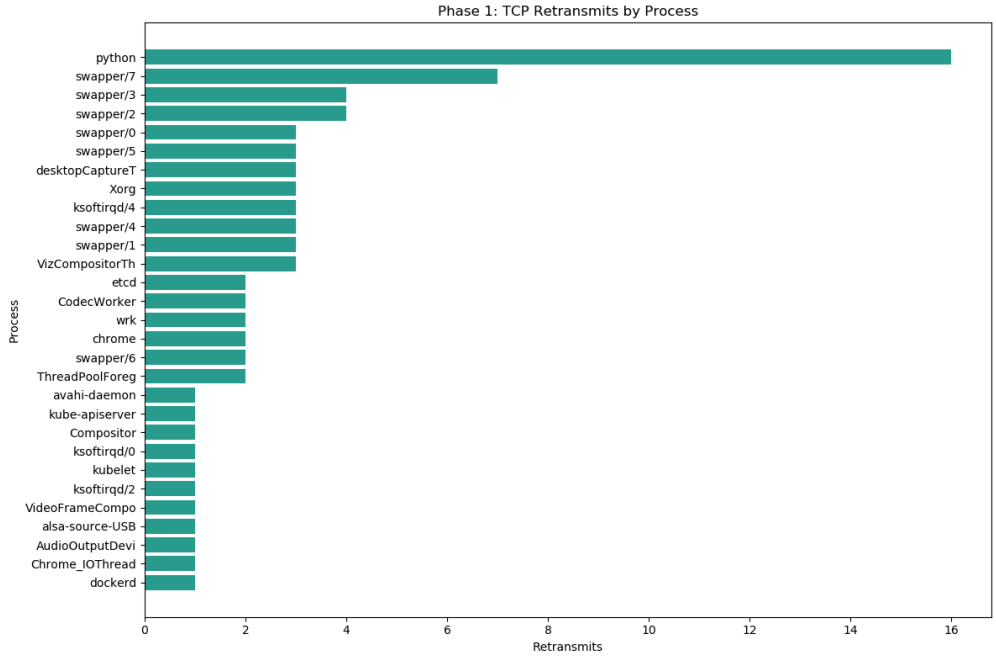
Observation

- In Phase 1 (Baseline), most scheduling delays are concentrated in lower latency buckets, indicating that processes are getting CPU time quickly with minimal waiting in the run queue.
- In Phase 2 (Stress), the distribution shifts significantly toward higher delay buckets, and the tail becomes much longer. This shows that threads are waiting longer in the run queue before execution, confirming CPU contention.
- The wider spread and heavier tail during stress indicate increased scheduling latency variability, which directly impacts request processing time and contributes to p99 latency spikes.
- In Phase 3 (Recovery), the distribution shifts back toward lower latency buckets, similar to the baseline, showing that CPU scheduling delays reduce once contention is removed.

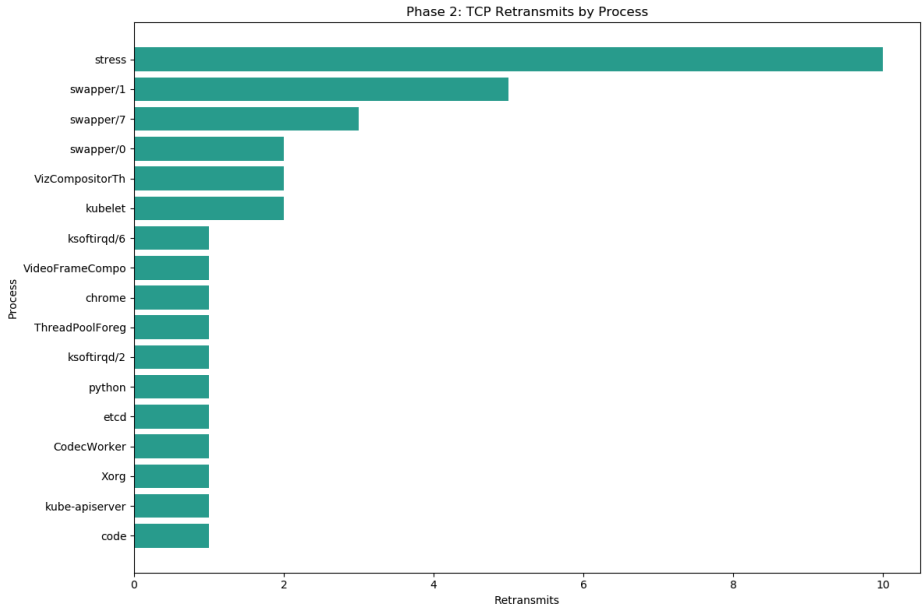
- However, a slight tail may still persist, indicating temporary backlog or delayed scheduler recovery.

3) TCP Retransmits Latency Probe

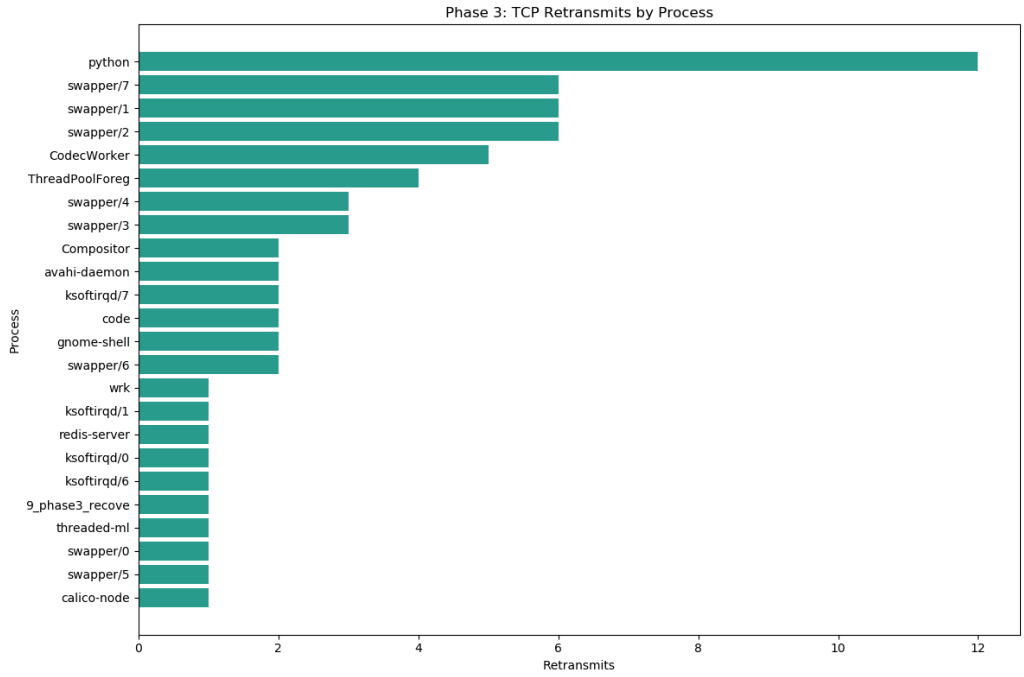
Phase 1: Baseline Plot



Phase 2: Stress Plot



Phase 3: Recovery Plot

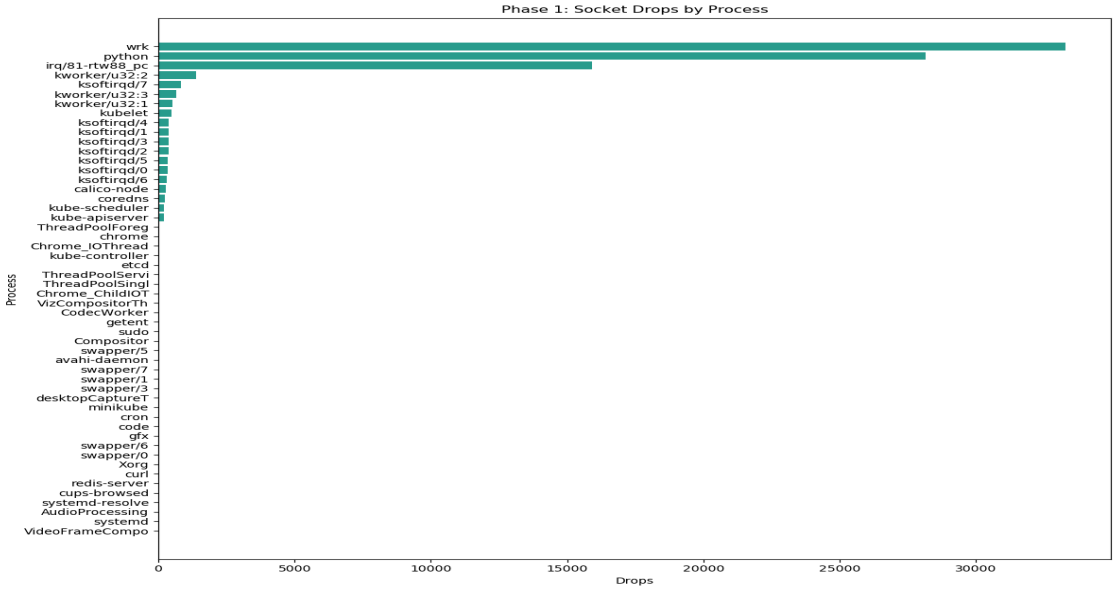


Observation

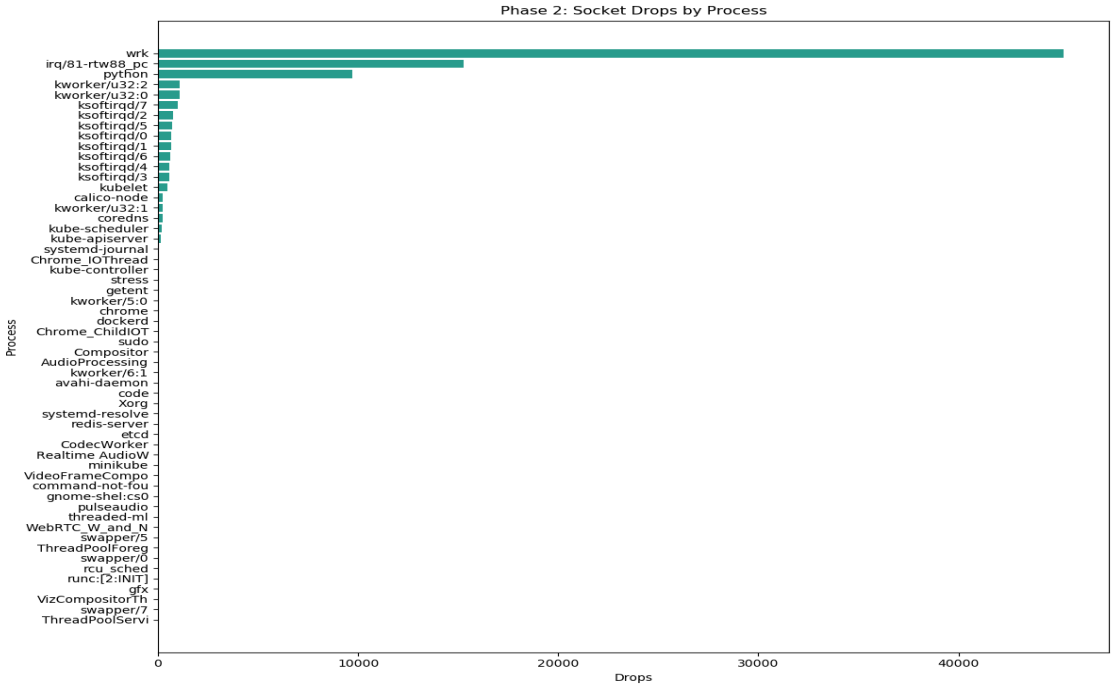
- In Phase 1 (Baseline), retransmissions are low and limited to a few processes, indicating stable network behavior with minimal packet loss or congestion.
- In Phase 2 (Stress), retransmissions increase significantly, especially for stress-related processes. This indicates network instability under load, likely due to buffer overflows, congestion, or delayed packet acknowledgments.
- The concentration of retransmits in specific processes shows that contention is not uniform, and certain services are more affected, contributing to inconsistent request latency.
- Increased retransmissions directly lead to longer request completion times, as packets must be resent, thereby amplifying tail latency (p99 spikes).
- In Phase 3 (Recovery), retransmissions reduce compared to the stress phase, indicating improved network conditions.
- However, retransmission counts are still slightly higher than baseline, suggesting residual congestion or delayed network recovery.

4) Socket Drop Probe Per Process

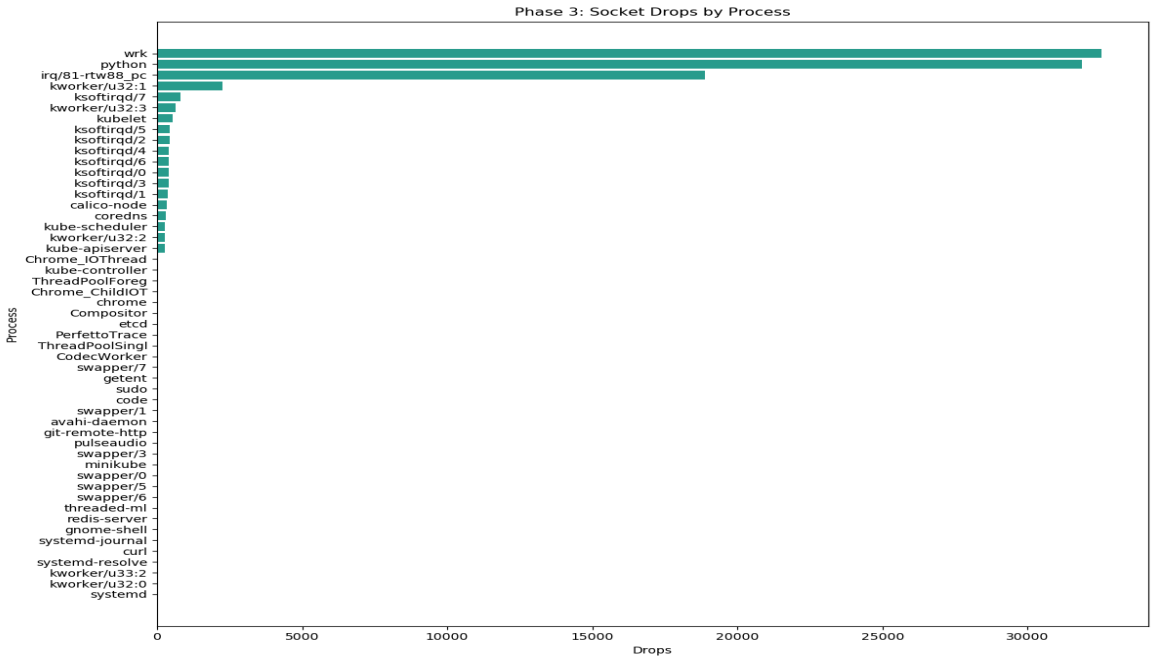
Phase 1: Baseline Plot



Phase 2: Stress Plot



Phase 3: Recovery Plot

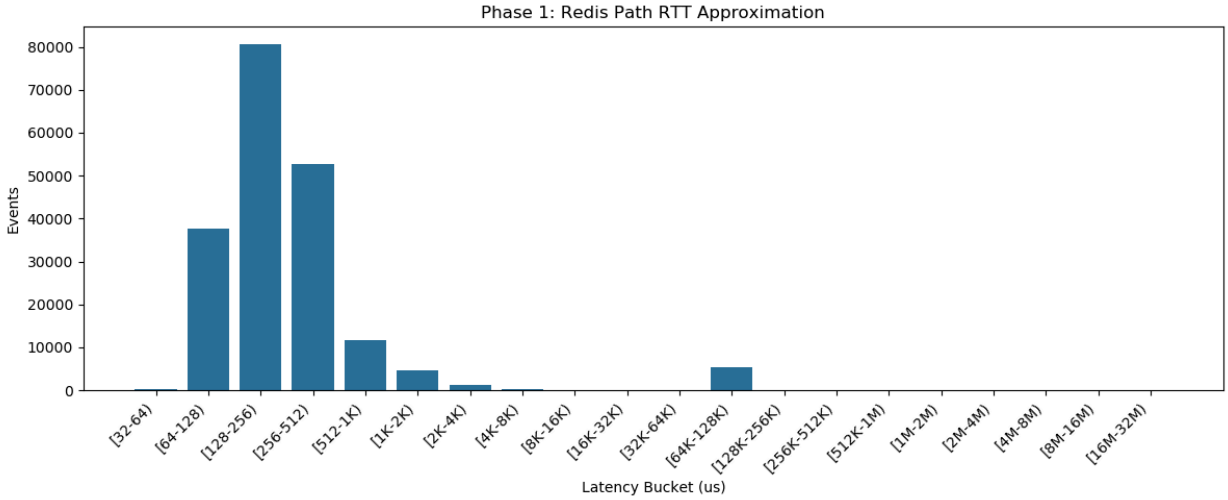


Observation

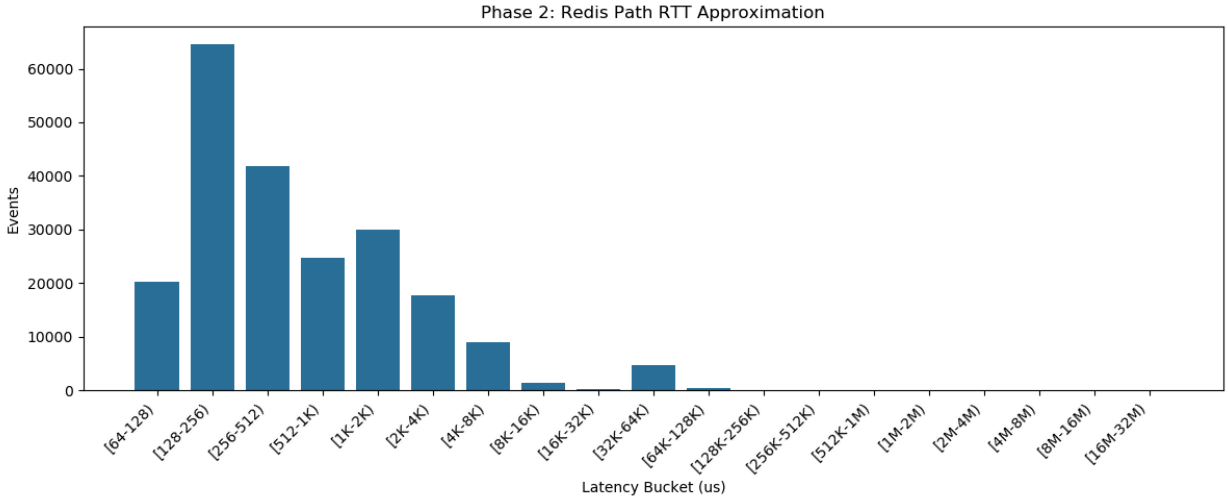
- In Phase 1 (Baseline), socket drops are present but relatively limited, concentrated in a few processes. This indicates minor buffering pressure but overall stable network behavior.
- In Phase 2 (Stress), socket drops increase drastically, with a sharp spike in specific processes. This clearly shows socket buffer overflow under high load, where the system is unable to handle incoming/outgoing packets efficiently.
- The concentration of drops in a few services indicates bottlenecked components, which leads to packet loss and request delays, directly contributing to tail latency spikes (p99).
- High socket drops imply that packets are being discarded before processing, forcing retransmissions and increasing end-to-end latency.
- In Phase 3 (Recovery), socket drops reduce compared to the stress phase, indicating that the system regains stability once contention is removed.
- However, drops may still be slightly higher than baseline, suggesting residual buffer pressure or incomplete recovery.

5) Redis Latency Probe

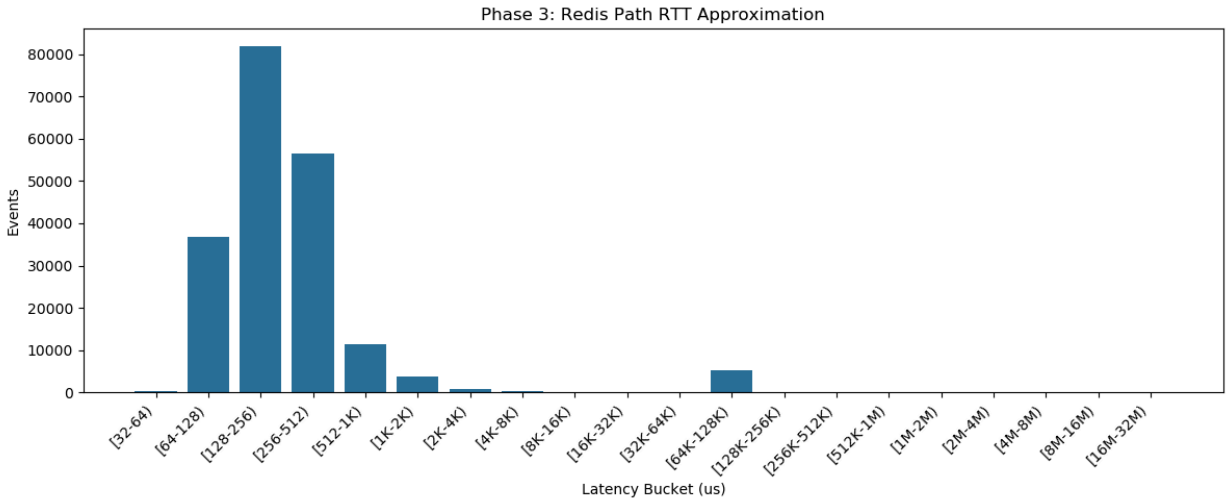
Phase 1: Baseline Plot



Phase 2: Stress Plot



Phase 3: Recovery Plot



Observation

- In Phase 1 (Baseline), Redis RTT values are mostly concentrated in lower latency buckets, indicating fast request-response cycles and minimal backend delay.
- In Phase 2 (Stress), the distribution shifts toward higher latency buckets with a noticeable spread, showing that Redis requests are taking longer to complete under load.
- The wider distribution and presence of higher latency buckets indicate increased queuing delay, network overhead, or CPU contention affecting Redis operations.
- This increase in backend latency contributes directly to end-to-end request delay, amplifying tail latency (p99) observed at the application level.
- In Phase 3 (Recovery), the distribution shifts back toward lower latency buckets, indicating that Redis performance improves once stress is removed.
- However, the slight presence of higher latency buckets suggests residual effects such as queue backlog or delayed recovery in the system.

10) Outcomes and Conclusions

Correlating Pod-Level SLO violations with kernel-level events

Define SLO

Typical industry SLOs for a web microservice:

SLO	Target
p99 latency	≤ 100 ms
p50 latency	≤ 50 ms
Throughput	≥ 500 rps

Mapping these against our data:

Phase 1:

Connections	P50(ms)	P99(ms)	p50 SLO (≤50ms)	p99 SLO (≤100ms)
10	5.43	91.34	Pass	Pass
50	125.67	198.16	VIOLATED	VIOLATED
100	301.96	394.38	VIOLATED	VIOLATED
200	607.18	705.38	VIOLATED	VIOLATED
300	908.58	1160.00	VIOLATED	VIOLATED

4 out of 5 load points violate at least one SLO. These are your "pod-level SLO violations" — concrete, measurable failures.

Phase 2:

Connections	P50(ms)	P99(ms)	p50 SLO (≤50ms)	p99 SLO (≤100ms)
10	12.64	76.55	Pass	Pass
50	126.02	190.85	VIOLATED	VIOLATED
100	289.44	607.79	VIOLATED	VIOLATED
200	578.39	668.14	VIOLATED	VIOLATED
300	827.69	926.63	VIOLATED	VIOLATED

Phase 3:

Connections	P50(ms)	P99(ms)	p50 SLO (≤50ms)	p99 SLO (≤100ms)
10	5.48	90.76	Pass	Pass
50	115.31	198.93	VIOLATED	VIOLATED
100	300.92	393.06	VIOLATED	VIOLATED
200	603.13	705.21	VIOLATED	VIOLATED
300	902.63	1110.00	VIOLATED	VIOLATED

The Correlation Table

We correlate by observing that at the same load level where p99 latency increases, corresponding eBPF probes show increased kernel-level delays such as scheduling latency, TCP send delay, retransmissions, and socket drops. This co-occurrence across phases establishes a causal relationship between kernel-level contention and tail latency inflation.

Phase	Load	SLO Violation	eBPF Signal	Interpreation
Phase 1	50	p99 violated	Mild runqlat + TCP delay	Early queuing and initial contention begins
Phase 1	100-300	Both violated	Gradual increase in runqlat + TCP latency	Resource contention builds with concurrence
Phase 2	50	Both violated	Increased runqla	Increased runqla
Phase 2	100	p99 spike	Runqlat tail + TCP send latency	CPU scheduling delay causes tail latency spike + network stack delay
Phase 2	100-300	Both violated	socket drops + Redis RTT	Buffer overflow causes packet drops + increases backend latency
Phase 3	50-300	Both violated	Signals reduced but present	System stabilizes after stress removal, but residual queueing and buffering effects persist

Proposed Mitigation Strategies

The correlation analysis in Section 3 surfaces which kernel subsystem is responsible for each SLO violation. We propose four mitigation strategies,each targeting a specific subsystem.

Mitigation 1: CPU Isolation

Targets: CPU scheduling delay (runqlat)
Give the application pod dedicated CPU cores
Prevents interference from other workloads (noisy neighbors)
Effect:

- CPU wait time reduces
- p99 latency improves significantly

Trade-off:

- Lower resource utilization (cores reserved even if idle)

Mitigation 2: Pod anti-affinity

Targets: Avoids CPU contention

- Prevent stress workloads from running on the same node

- Ensures app and stress pods are separated

Effect:

- Reduces chance of contentio
- Improves stability

Works best when combined with CPU pinning

Mitigation 3: Priority based scheduling

Targets: survival under node pressure, not latency directly.

Mechanism: a high-value `PriorityClass` is assigned to nginx, and low-priority or BestEffort classes are assigned to batch workloads. Under memory or CPU pressure, the kubelet evicts low-priority pods first.

Expected effect: during node overload, nginx survives while best-effort workloads are shed. Complements the QoS mitigation by covering the case where the source of contention is not another pod but the node itself being oversubscribed.

Key Outcomes

The research was able to create an experimentally verifiable test bed environment, employing microservices architecture, Kubernetes and Minikube, and conduct experiments across three different stages: baseline, stress, and recovery.
Instrumentation at the kernel level was achieved via the usage of eBPF/bpftrace to observe scheduling and network activity within the running application.
It is clear from experiments conducted on the created infrastructure that CPU contention results in increased latencies for scheduling operations, thus contributing directly to increase in latencies for requests, especially when measuring higher percentiles (p99). Moreover, higher concurrency in the system correlates with increased variance of latencies for requests, pointing to the effect of queuing and resource competition.
By utilizing eBPF instrumentation, the researcher was able to get insights about run-queue latencies, TCP delays and retransmissions, as well as dropped sockets, that otherwise would be hidden by application-level profiling.

Conclusion

Based on the experiments' findings, it can be said that tail latency in microservices is highly impacted by system-level factors such as scheduling latency and network stack behavior. Although standard monitoring provides performance metrics at the application level, such tools do not suffice in pinpointing the reasons behind the latency problems. With the use of eBPF, the ability to instrument the kernel and perform low-latency monitoring is available, making it easier to find the correlations between SLO violation problems and the system-level causes of those problems.

Future Work

Future extensions of this work can focus on expanding the experimental setup to multi-node Kubernetes environments on physical machines to study inter-node effects. Further experiments can also explore more complex microservice architectures and evaluate the impact of different resource allocation strategies and scheduling policies on latency behavior.

11) References

- Kubernetes Documentation https://kubernetes.io/docs
- eBPF Documentation https://ebpf.io
- Cilium Documentation https://docs.cilium.io
- bpftrace Documentation https://github.com/iovisor/bpftrace
- Minikube Documentation https://minikube.sigs.k8s.io/docs